

Project Documentation: AI-Powered Role-Based Candidate Screening System

This document explains the system in depth: what it does, why it's built the way it is, and how data moves through it end to end. It complements [README.md](#) (setup instructions, quick design-decision summary) and [DEMO.md](#) (video recording checklist) rather than repeating them — read this one if you want to actually understand how the system works, not just how to run it.

1. What this system does

This is a simulated technical interviewer. A candidate uploads their resume and picks a target role (AI/ML Engineer, Backend Engineer, or Data Scientist). The system then:

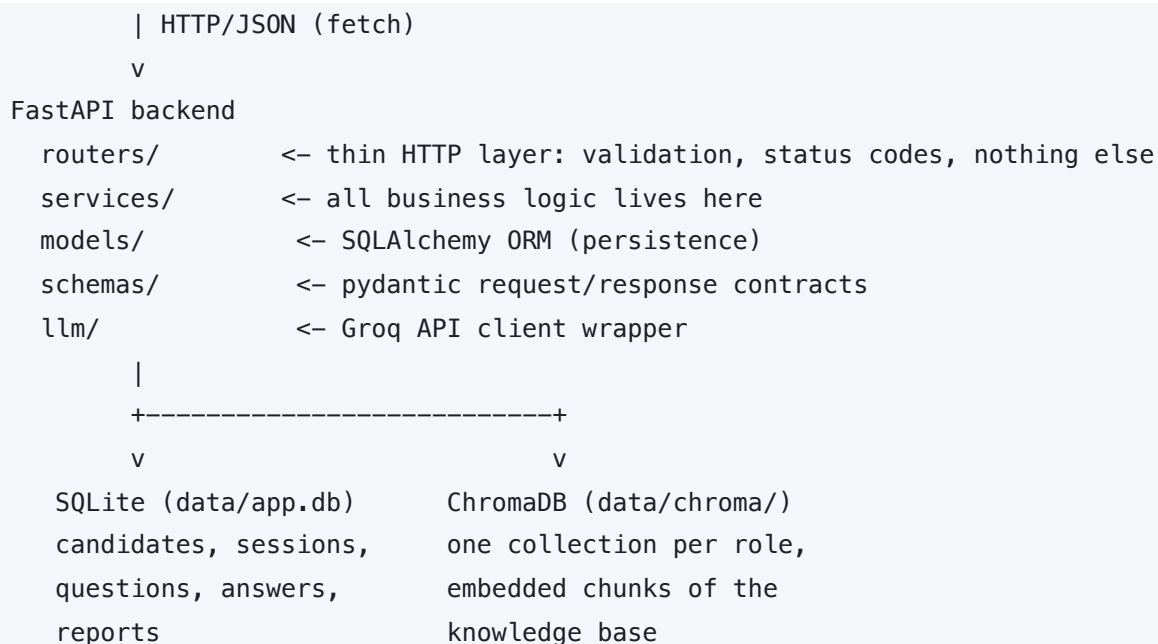
1. Reads the resume and figures out what the candidate actually knows (skills, technologies, rough seniority level).
2. Picks a topic to probe, grounded in a role-specific knowledge base, using Retrieval-Augmented Generation (RAG) — not a fixed question bank.
3. Asks a question that's provably generated from real reference material, not invented by the model from nothing.
4. Scores the candidate's answer against that same reference material.
5. Decides what to ask next based on how the candidate did: a weak answer gets a simpler follow-up on the same topic; a strong answer moves on to a new topic.
6. After five questions, produces a structured report: overall score, narrative summary, strengths, gaps, and a full transcript.

The core idea the assignment is testing is whether the questions are *actually* generated from the candidate + role + knowledge base — not templated, not generic, and traceable back to the exact source material that produced them. Every question in this system stores the retrieval query and the exact retrieved text chunks that grounded it, and the UI exposes this directly via a "Why this question?" panel.

2. High-level architecture

React frontend (Vite, JSX)

|



Why this split. Routers are deliberately "dumb" — they parse the request, call exactly one service function, and translate the result (or exception) into an HTTP response. All the actual thinking — how to build a retrieval query, how to score an answer, when to advance to a new topic — lives in `services/`, where it can be unit-tested without spinning up FastAPI at all. This is what let the test suite catch real bugs (see section 7) by testing `query_builder.py` and `rag_pipeline.py` directly, independent of the HTTP layer.

3. The pipeline, step by step

The assignment asks for a specific pipeline shape: **Context → Question → Answer → Storage**. Here is exactly how each stage works in this codebase.

3.1 Candidate entry and resume processing

`routers/candidates.py` accepts a resume file (PDF or `.txt`) and a target role.
`services/resume_parser.py` does the actual work:

- **Text extraction:** `pypdf` pulls raw text out of a PDF; plain text files are read directly.
- **Skill/technology matching:** rather than running a full NLP/NER model (heavyweight, slow cold-start, hard to unit-test), the parser matches the resume text against a curated keyword taxonomy (`services/skills_taxonomy.py`) covering programming languages, ML/AI terms, data science tools, backend technologies, and cloud/DevOps tools. Matching is case-insensitive and word-boundary-aware — critically, *single-character* terms like "C" or "R" use a stricter boundary that excludes `+`, `#`, and `.` as valid boundary characters, because the looser boundary was found to spuriously match "C" inside "C++", "C#", or abbreviations like "C.S." (a real bug found via manual testing — see section 7).

- **Experience-level inference:** a regex looks for explicit "N years of experience" phrasing, and falls back to scanning for seniority keywords ("senior", "intern", "lead", etc.) if no explicit number is found. This produces a coarse `junior / mid / senior` label that later influences question difficulty.

The result — resume text, matched skills, matched technologies, and the inferred experience level — is stored as a `Candidate` row and used for everything downstream.

3.2 Context construction (dynamic query building)

`services/query_builder.py` decides what to actually ask the retrieval system next. This is the piece that makes the system's questions depend on *this specific candidate*, not just the role in the abstract.

- **First question:** `opening_query()` takes the candidate's role-relevant skills (the intersection of what they listed and what matters for the chosen role) and builds a retrieval query around their single strongest matching skill. Earlier, this combined the top three skills into one query string, but that measurably hurt retrieval precision — a multi-skill query would sometimes rank a chunk about a *different* skill above the one actually being asked about, which looked incoherent downstream. Narrowing to one skill per query fixed this; the full skill list is still passed separately to the question-generation prompt, so nothing is lost.
- **Subsequent questions:** `next_query()` looks at the previous question's topic and the score it received. A score below 3/5 re-probes the *same* topic with a query asking for a more fundamental version of the question. A score of 3+ moves on to the next role-relevant skill the candidate hasn't been asked about yet (case-insensitive comparison against a running list of covered topics), or — once every matched skill has been covered — asks for something "beyond" what's been covered, still role-appropriate.

This adaptivity (assignment section 3, called out as "optional but valuable") was specifically stress-tested: feeding deliberately weak or mismatched answers correctly kept the interview re-probing the same topic rather than advancing, and feeding strong, broad answers correctly advanced through five distinct topics without repeats.

3.3 Knowledge retrieval (RAG)

This is the assignment's core focus, so it gets the most detail.

Ingestion (`scripts/ingest_kb.py` → `rag_pipeline.ingest_role()`):

1. Every `.md`, `.txt`, and `.pdf` file under `backend/knowledge_base/<role>/` is loaded (PDFs via `pypdf`, same extraction path as resumes).
2. **Semantic chunking**, not fixed-size chunking. Each document is split into sentences, each sentence is embedded, and a new chunk boundary is placed exactly where cosine similarity between consecutive sentence embeddings drops below a threshold (0.55 by default) — i.e. where the topic actually shifts. `MIN_CHUNK_CHARS` / `MAX_CHUNK_CHARS` bound the result so

neither a long run of similar sentences produces one giant chunk, nor a run of dissimilar short sentences produces many context-poor slivers.

Why not a fixed character count? An earlier version of this pipeline did exactly that (with sentence-boundary snapping so it at least didn't cut mid-sentence). It had a real, measured weakness: a fixed budget has no concept of where one idea ends and the next begins, so it routinely grouped unrelated sentences into one chunk and split single ideas across two overlapping chunks. This dilutes both the embedding's signal and the context handed to the question generator — a specific instance of what's sometimes called "context rot" in RAG systems: stuffing a prompt with loosely-related or redundant text degrades how well an LLM can attend to what actually matters. 3. Each finished chunk is embedded (`sentence-transformers/all-MiniLM-L6-v2` , run locally, not via a hosted API — embeddings are needed on every single retrieval call, not just at ingestion, so keeping that hot path free of network latency and per-call cost mattered more than the marginal quality gain of a larger hosted model) and stored in a per-role, persistent Chroma collection, upserted (not added) by a deterministic ID so re-running ingestion after editing a KB file is idempotent. 4. Chroma enforces a hard maximum batch size per `upsert()` call (5461 in the installed version). A handful of short markdown articles never approached this, but a full textbook PDF's chunk count did (a real ingestion failure — "Batch size of 6343 is greater than max batch size of 5461" — was hit and fixed once real textbook PDFs were added). Ingestion now batches upserts in groups of 5000 regardless of corpus size.

Retrieval (`rag_pipeline.retrieve()`):

1. The query text from `query_builder` is embedded with the same model.
2. Chroma's approximate-nearest-neighbor search (cosine distance) returns the top-k candidate chunks.
3. **Context-rot mitigation on the retrieval side too:** any result below a minimum similarity threshold (0.15 by default) is dropped rather than force-included just to fill the quota — a fixed top-k can otherwise pad the LLM prompt with a barely-relevant chunk that then gets treated as if it were meaningful grounding. At least one chunk (the single best match) is always kept, so a question is never generated with zero context.
4. **Fail loudly, not silently, if a role was never ingested.** If a role's Chroma collection is completely empty (ingestion was simply never run), `retrieve()` raises `KnowledgeBaseNotIngestedError` with the exact fix command, and the API returns a 503 with that message. The earlier behavior — returning an empty list — was indistinguishable downstream from a normal (if unusually low-relevance) retrieval, meaning a skipped ingestion step could silently degrade the whole "RAG" system into a non-RAG one with no visible signal to anyone testing the app.

3.4 Question generation

`services/question_generator.py` takes the retrieved chunks, the candidate's experience level, their matched skills, and (if this isn't the first question) the previous question/answer/score, and asks an LLM (Groq, `llama-3.3-70b-versatile`) to produce exactly one question. The prompt explicitly requires the question to be answerable *only* from the retrieved excerpts, forbids generic

"what is X?" phrasing, and asks the model to calibrate difficulty and adapt based on the previous score. This is what makes the output "RAG" rather than "ask an LLM for interview questions" — the model is constrained to reason from the specific retrieved text, not free-associate from its own training data.

A deterministic fallback exists for when no `GR00_API_KEY` is configured: it picks the top retrieved chunk, extracts one clean sentence from it (with markdown headings stripped and sentence-splitting that handles both period-space and period-newline boundaries — the KB's own prose wraps with newlines after most sentences, which an earlier, naive splitter didn't handle, and ended up quoting almost an entire chunk verbatim including its raw heading), and builds a templated question around it. This keeps the system fully functional end to end with zero cost/API dependency, but it is explicitly a lower-quality path — the real LLM path is the intended experience.

A note on topic labeling. Whatever the LLM (or the fallback) returns as the question's "topic" is deliberately overridden back to the exact skill-level string that `query_builder` used to build the retrieval query, run through a small display-formatting function (`skills_taxonomy.display_label()`) that renders raw taxonomy tokens naturally (`c` → `C`, `rest api` → `REST API`, `fastapi` → `FastAPI`, etc.). This exists because of two real bugs found via testing: (1) letting the fallback path relabel the topic based on whichever chunk it happened to quote broke the topic-coverage tracking described in 3.2, since the tracking depends on consistent topic strings; and (2) a candidate's resume mentioning "C++" spuriously matched the bare single-letter token "c" in the skills taxonomy, which then got quoted raw in a generated question ("...explain how this relates to c...") before the display-label fix.

3.5 Interactive interview and response handling

`services/interview_engine.py` is the single orchestrator that ties all of the above together and is the only thing routers call for interview-lifecycle logic. `start_session()` creates an `InterviewSession` row and its first `Question`. `submit_answer()`:

1. Scores the answer (`question_generator.evaluate_answer()`) — same LLM-with-fallback structure as question generation, grading 1–5 against the same retrieved context the question was built from) and stores it as an `Answer` row.
2. Advances the session's `current_index`.
3. Either generates the next adaptive question, or — once the configured question count (5 by default) is reached — marks the session complete and triggers report generation.

All of this happens within one request; there's no separate polling step for "is my answer graded yet."

3.6 Final output

`services/report_generator.py` reads every stored question/answer pair for a completed session and produces: an overall average score, a per-topic score breakdown, and (via the LLM, with a deterministic aggregation fallback) a narrative summary plus concrete strengths and improvement

areas — each meant to reference specific topics, not generic praise or criticism. The frontend's summary page renders all of this plus the full transcript.

4. Data model

Five tables, each existing for a clear reason:

- **Candidate** — resume text, extracted skills/technologies, inferred experience signals, target role.
- **InterviewSession** — links to a candidate; tracks role, status (`in_progress` / `completed`), how many questions have been asked, and when it started/finished.
- **Question** — the generated question text, its topic and difficulty, and critically, `retrieval_query` and `retrieved_context` (the exact query and chunks that produced it) as JSON — this is what makes every question traceable after the fact, not just at generation time.
- **Answer** — one per question, storing the candidate's text, score, and feedback.
- **Report** — one per completed session: overall score, summary text, topic breakdown, strengths, improvement areas.

SQLite was chosen deliberately over Postgres: persistence is required by the assignment, but requiring an external database service just to run the app locally would add friction with no benefit at this scale. Every model uses portable SQLAlchemy types, so switching `DATABASE_URL` to Postgres later requires no other code change.

5. Frontend

React (JSX, no TypeScript) + Vite, structured as a strict three-stage wizard — `UploadPage` → `InterviewPage` → `SummaryPage` — with a single `stage` variable in `App.jsx` deciding which one renders. No router library is used: the flow is linear and never needs deep-linking or back/forward navigation between stages, so URL-based routing would be pure overhead.

Notable details:

- `InterviewPage` 's "Why this question?" toggle renders the actual retrieved chunks (source file, similarity score, text) and the retrieval query — the RAG traceability from section 3.3, made visible in the UI, not just present in the database.
- Focus management: since there's no router-driven page navigation, a `useEffect` in `App.jsx` moves focus to the main content region on every stage change (with `aria-live="polite"`), so a screen reader user gets a signal that the content changed — otherwise the wizard would silently swap entire page components under them.

- The answer form is fully unmounted (not just disabled) while a score/feedback flash is showing between questions, which structurally prevents double-submission during that window rather than relying on a disabled attribute alone.
-

6. Why Groq, and what happens without it

Question generation and answer scoring both call Groq's `llama-3.3-70b-versatile` model through a single narrow wrapper (`llm/groq_client.py`) — every other module depends on that one function signature (`chat_json(system_prompt, user_prompt) -> dict`), so switching providers later means changing one file. The client has an explicit 15 second timeout, so a hung request degrades to the fallback path instead of occupying a request thread indefinitely.

If `GROQ_API_KEY` is unset, `is_llm_configured()` returns false and every LLM call site falls through to its deterministic counterpart described in 3.4/3.5 — the system never crashes or blocks on a missing key, it just runs in a visibly lower-quality mode.

7. What testing actually caught (and why it matters)

The test suite (18 tests, ~3 seconds, `backend/tests/`) is structured in three layers: unit tests for resume parsing and query building, an end-to-end HTTP-level test of the full candidate → session → interview → report flow, and dedicated tests for the fail-loudly-on-missing-ingestion behavior. A few of these tests exist specifically *because* they caught real bugs during development, not as a checklist exercise:

- **Topic-repetition bug:** the fallback question generator originally relabeled a question's topic based on whichever KB chunk the retriever ranked highest, instead of the topic that was actually being asked about. This silently broke the adaptive topic-coverage logic — the interview kept re-asking about the same article regardless of the intended topic. `test_query_builder.py`'s topic-advancement test and `test_api_flow.py`'s topic-distinctness assertion both guard against this regressing.
- **Blocking event loop:** an independent backend architecture review (spawned as a judge agent specifically to evaluate the finished system) found that the resume-upload route was declared `async def` while doing blocking work (PDF text extraction, synchronous database commits) directly on the event loop — every other route was correctly plain `def`, letting FastAPI run it in a worker thread. Fixed and verified.
- **Test/production environment leakage:** after a real `GROQ_API_KEY` was added to `backend/.env` for local development, the test suite silently started exercising the real (non-deterministic, paid) LLM instead of the fallback logic it's designed to test — because pydantic-settings reads `.env` directly and nothing in the original test fixture overrode that. This looked

exactly like a scoring regression before the actual cause was found. `conftest.py` now explicitly forces `GR0Q_API_KEY=""` for every test.

- **Chroma test isolation:** the test suite originally depended on whatever the developer happened to have already ingested locally. It now ingests an isolated, temporary Chroma store once per test session from the real `.md / .txt` knowledge base articles (explicitly excluding large PDF textbooks, so adding a full book to the real knowledge base doesn't balloon test runtime).

8. Knowledge base composition and sourcing

`backend/knowledge_base/` has three role directories. Content in each is a mix of original technical writing and, for `ai_ml_engineer` and `data_scientist`, real textbook PDFs:

- **Original articles** (22 total: 8 AI/ML Engineer, 8 Backend Engineer, 6 Data Scientist) — written to mirror the depth and topic structure of the assignment's suggested references (e.g. the AI/ML articles mirror Mitchell's table of contents: concept learning, decision trees, neural networks, Bayesian learning, instance-based learning, computational learning theory, reinforcement learning, hypothesis evaluation).
- **Two textbooks committed to git:** Mitchell's *Machine Learning* and Bishop's *Pattern Recognition and Machine Learning*, both fetched directly from the author's/publisher's own official free hosting (Mitchell's own CMU page; Microsoft Research's official PRML page) — both verified to contain real, extractable book content before being added.
- **Five textbooks present locally only, excluded from git:** the remaining books named across the assignment's role categories (Burkov's Hundred-Page ML Book, ML for Absolute Beginners, an AI/ML/Deep Learning overview, Muller & Guido's Intro to ML with Python, Brownlee's Master ML Algorithms). The assignment's own linked URLs for these resolve to what are clearly unauthorized mirrors of commercially-sold books — a well-known piracy aggregator, a copy with `z-lib.org` literally in its filename, and other obscure mirror sites, none of them official distribution channels. They were sourced locally (not fetched by this project) and are explicitly listed in `.gitignore` by exact filename so they enrich local demoing without this public repository ever hosting or redistributing them. See README's "Using the assigned textbook PDFs" section for the full per-book breakdown.

This is a considered tradeoff, not an oversight: the assignment's requirement is satisfied (real, authoritative textbook content grounds the RAG pipeline for the roles it was assigned to), while stopping short of what would amount to republishing pirated commercial books to the public internet via git.

9. Known limitations (by design, not by accident)

- **No authentication.** Any client can view any candidate/session/report by guessing a sequential ID. Fine for a single-user local demo; would need real access control before holding real users' data.
 - **No rate limiting.** A public deployment could be hit in a loop to drive unbounded Groq API cost. Out of scope for a local demo.
 - **Fallback scoring is a blunt heuristic** (answer length + keyword overlap), not real reasoning — it exists purely so the system runs with zero API key configured, and will score a correct-but-differently-worded answer unfairly low. The real Groq-backed path is the intended experience.
 - **Ingestion is a required, explicit, manual step**, deliberately not automated inside a request handler — see section 3.3's fail-loudly behavior.
-

10. Summary

The system's design consistently optimizes for one thing: every generated question should be provably grounded in real, retrievable source material, specific to this candidate and this role, and every claim the system makes about "why" a question was asked should be checkable — in the database, in the API response, and in the UI. Where a shortcut was taken (the deterministic fallback path, the original-written articles standing in for some textbooks), it's documented as a shortcut with its reasoning, not presented as equivalent to the real thing.